Space-Aware Proxy Selection for Training-Free Dense Prediction NAS

Anonymous CVPR submission

Paper ID *****

Abstract

Zero-cost proxies score untrained networks in seconds, enabling efficient neural architecture search (NAS), yet no single proxy works well across all search space types. We show that space-aware proxy selection*, a simple strategy that assigns different proxies to macro-level and micro-level search spaces, substantially outperforms any individual proxy for dense prediction tasks. Evaluating 7,344 architectures across five tasks on TransNAS-Bench-101 with real trained ground truth, our Space-Aware selector achieves Spearman* ρ=0.631 *for semantic segmentation, a* ***+49% improvement*** *over the best single proxy (GradNorm,* ρ=0.423*). Similar gains appear for surface normal prediction (+24%), object classification (+16%), and scene classification (+22%). The selector is enabled by two observations: (1) our proposed* Multi-Scale Feature Separability *(*MSFS*) proxy achieves* ρ=0.706 *for segmentation within macro search spaces, outperforming GradNorm (*ρ=0.589*), but is weak in micro spaces where capacity proxies dominate; and (2) z-score normalization within each space removes inter-space scale artifacts that otherwise destroy cross-space correlation. NAS simulation experiments confirm practical value: at* K=10*,* MSFS *finds macro-space architectures with mean mIoU of 31.00, matching or exceeding GradNorm (30.81). Architectural analysis reveals that* MSFS *captures information beyond simple depth and capacity, predicting segmentation quality within same-depth architecture groups (*ρ=0.43–0.65*). We also report* Spatial Feature Consistency *(*SFC*) as a negative result with detailed failure analysis. All experiments run on a single A10G GPU in under one hour.*

1. Introduction

Neural architecture search (NAS) has demonstrated that automatically discovered architectures can match or exceed hand-designed ones for image classification [6], semantic segmentation [2, 14], and object detection [4, 7, 9]. However, conventional NAS requires training hundreds of candidates, often consuming thousands of GPU-hours. **Zero-cost proxies** score untrained networks through a single forward or backward pass, reducing the search from days to seconds. Methods such as NASWOT [15], SynFlow [17], GradNorm [1], TE-NAS [3], and ZenNAS [12] achieve strong rank correlations with trained accuracy on classification benchmarks.

A critical problem remains: *no single proxy dominates across all search spaces and tasks.* NAS-Bench-Suite-Zero [10] showed this across 28 tasks, and TransNAS-Bench-101 [8] confirmed that optimal architectures are task-specific. Our own analysis reveals a more specific phenomenon: proxy effectiveness depends not only on the *task* but critically on the *type of search space*. In macro spaces that vary depth, width, and connectivity, task-specific proxies excel. In micro spaces that vary internal cell operations, capacity-based proxies dominate. Mixing these two regimes in a single evaluation destroys the signal of specialized proxies.

This paper makes three contributions:

1. We propose **Space-Aware Proxy Selection**, a strategy that assigns different proxies to different search space types: MSFS for macro-like spaces and the best capacity proxy (#Params or SynFlow) for micro-like spaces, combining scores via z-score normalization. This simple approach achieves ρ=0.631 for segmentation across all 7,344 architectures, a 49% improvement over GradNorm (ρ=0.423), with consistent gains across four of five tasks.
2. We introduce MSFS (*Multi-Scale Feature Separability*), a training-free proxy that quantifies inter-scale feature divergence from a single forward pass on random input. MSFS achieves ρ=0.706 for segmentation in the macro search space of TransNAS-Bench-101, outperforming GradNorm (ρ=0.589). Architectural analysis shows that MSFS captures information beyond depth and capacity, predicting quality within same-depth groups (ρ=0.43–0.65).
3. We validate all results against **real trained ground**

truth on TransNAS-Bench-101 (7,344 architectures, 5 tasks), including NAS simulation with top-K selection that confirms practical search utility. We also provide a transparent negative result analysis for SFC (*Spatial Feature Consistency*).

The space-aware insight is both simple and general: since NAS practitioners design their search spaces and therefore know whether they are varying macro-level or micro-level parameters, selecting the appropriate proxy per space type is essentially free.

2. Related Work

Zero-cost NAS proxies. Mellor et al. [15] proposed NASWOT, scoring architectures by ReLU activation pattern diversity. Tanaka et al. [17] introduced SynFlow, a data-free pruning score used as a NAS proxy. Abdelfattah et al. [1] unified multiple proxies under a common framework and showed that no single proxy dominates across all benchmarks. TE-NAS [3] uses NTK spectra, and ZenNAS [12] uses Gaussian complexity. All were designed for and validated primarily on classification. We evaluate TE-NAS, ZenNAS, and a fixed version of NASWOT on TransNAS-Bench-101 and find that ZenNAS exhibits the same space-dependent behavior as MSFS: strong in macro space (ρ=0.751 for segmentation) but weak in micro space (ρ=0.097), further supporting the space-aware thesis.

NAS for dense prediction. Auto-DeepLab [14] searches hierarchical architectures for segmentation. NAS-FPN [9] and SpineNet [7] discover feature pyramid topologies. DetNAS [4] trains detection-specific supernets. FasterSeg [2] and SqueezeNAS [16] target real-time segmentation. All require expensive search with actual training.

Proxy transferability and benchmarks. NAS-Bench-Suite-Zero [10] evaluated 13 proxies across 28 tasks, revealing poor cross-task transferability. TransNAS-Bench-101 [8] provides trained performance for 7,352 architectures across seven vision tasks in two search spaces (macro and micro), enabling rigorous cross-task and cross-space evaluation. Xiang et al. [18] integrated zero-cost scores into differentiable NAS. LPZero [11] extended proxy discovery to NLP. To our knowledge, no prior work has proposed *space-aware* proxy selection that adapts the proxy choice to the search space type.

3. Method

We propose two task-specific zero-cost proxies (Secs. 3.1 and 3.2) and a space-aware strategy for combining them with existing proxies (Sec. 3.3). Given an untrained backbone f_θ producing feature maps $\{F_1, F_2, \ldots, F_L\}$ at L=4 scales (stem + 3 stages) from a random Gaussian input $\mathbf{x} \sim \mathcal{N}(0,1)$ of size 64×64, we define the following proxies.

3.1. Multi-Scale Feature Separability (MSFS)

Dense prediction relies on feature pyramids where different scales capture objects at different resolutions [13]. We hypothesize that architectures producing more diverse multi-scale representations at initialization yield better backbone features for dense tasks.

(1) Inter-Scale Divergence (ISD). For each pair of feature scales (l_i, l_j), we project their global-average-pooled representations to a common dimensionality and measure distribution divergence:

$$\mathrm{ISD}(l_i, l_j) = \|\mu_i - \mu_j\|_2 + \left|\log \frac{\sigma_i^2}{\sigma_j^2}\right| + \frac{\|C_i - C_j\|_F}{d} \quad (1)$$

where μ, σ^2, and C denote per-scale means, variances, and correlation matrices, and d is the common feature dimensionality.

(2) Scale Awareness (SA). We measure the product of spatial and channel variance at each level, capturing positional information preservation:

$$\mathrm{SA} = \log\left(\sum_{l=1}^{L} \mathrm{Var}_{h,w}[F_l] \cdot \mathrm{Var}_c[F_l]\right) \quad (2)$$

The combined proxy is $\mathrm{MSFS} = \delta \cdot \overline{\mathrm{ISD}} + \eta \cdot \widetilde{\mathrm{SA}}$, where δ=0.6, η=0.4, $\overline{\mathrm{ISD}}$ averages over all $\binom{L}{2}$ pairs, and $\widetilde{\mathrm{SA}}$ applies affine normalization $(\mathrm{SA}+10)/10$. These weights were fixed a priori and were *not* tuned on any evaluation data.

3.2. Spatial Feature Consistency (SFC)

We also design SFC, which measures spatial coherence properties hypothesized to be relevant for per-pixel prediction. SFC combines three components: Neighbor Coherence (NC), measuring cosine similarity between spatially adjacent feature patches; Boundary Sensitivity (BS), measuring spatial gradient magnitude relative to feature magnitude; and Spatial Diversity (SD), the log-variance of features across spatial dimensions. The combined proxy is $\mathrm{SFC} = \alpha \cdot \overline{\mathrm{NC}} + \beta \cdot \overline{\mathrm{BS}} + \gamma \cdot \widetilde{\mathrm{SD}}$, with α=0.4, β=0.35, γ=0.25, all fixed a priori. As we show in Sec. 4.6, SFC is a negative result.

3.3. Space-Aware Proxy Selection

Our key methodological contribution is the observation that proxy effectiveness is strongly search-space-dependent. Rather than seeking a single universal proxy, we propose selecting the best proxy for each search space type and combining via z-score normalization.

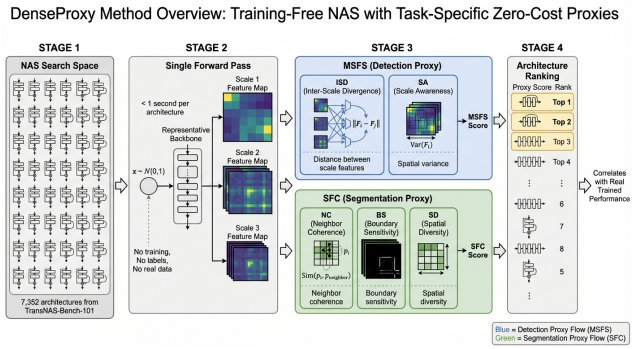

Figure 1. **Overview of DenseProxy.** Task-specific zero-cost proxies are computed from a single forward pass on random Gaussian input. The Space-Aware selector assigns MSFS to macro spaces and capacity proxies to micro spaces, then combines via z-score normalization.

Motivation. In macro spaces (varying depth, width, connectivity with fixed cell types), multi-scale structural properties dominate, and MSFS excels. In micro spaces (varying internal cell operations), raw representational capacity dominates, and #Params or SynFlow excels. When scores from these two regimes are naïvely combined, the different score distributions destroy cross-space correlation.

Procedure. Given architectures from S search spaces, we: (1) compute proxy scores p_i for each architecture i; (2) z-score normalize within each space: $\hat{p}_i = (p_i - \mu_s)/\sigma_s$, where μ_s and σ_s are the mean and standard deviation of scores within space s; and (3) assign the best proxy per space type (MSFS for macro-like, #Params or SynFlow for micro-like). The space type is known to the NAS practitioner, since they design the search space.

Fig. 1 illustrates the complete pipeline.

4. Experiments

4.1. Setup

Benchmark. We evaluate on TransNAS-Bench-101 [8], which provides real trained performance for 7,352 architectures across seven vision tasks. After filtering, we evaluate on N=7,344 architectures across five primary tasks: semantic segmentation (mIoU), surface normal prediction (SSIM), object classification (top-1), scene classification (top-1), and autoencoding (SSIM). TransNAS-Bench-101 contains two search spaces: the *macro* space (3,256 architectures based on ResNet-basic cells varying in depth, width, and connectivity) and the *micro* space (4,088 architectures based on custom micro cells). This distinction is central to our analysis.

Baselines. We compare against #Params (parameter count), GradNorm [1], and SynFlow [17]. NASWOT [15] is excluded because TransNAS-Bench-101 backbone architectures lack ReLU activations in the final layer, causing NASWOT to produce all-zero scores. We additionally evaluate TE-NAS [3] (NTK condition number), ZenNAS [12] (Gaussian complexity), and a fixed NASWOT variant using intermediate ReLU/BatchNorm activations (NASWOT-fixed) on N=6,590 successfully constructed architectures.

Table 1. **Per-search-space Spearman ρ for segmentation and normal prediction.** MSFS achieves ρ=0.706 for segmentation in macro space, outperforming all baselines. In micro space, capacity proxies (#Params, SynFlow) dominate.

	Macro (N=3,256)		Micro (N=4,088)	
Proxy	Seg. ρ	Normal ρ	Seg. ρ	Normal ρ
MSFS (Ours)	**0.706**	0.453	0.163	0.393
SFC (Ours)	−0.509	−0.580	0.219	0.469
GradNorm	0.589	**0.687**	0.207	0.418
SynFlow	−0.223	−0.301	0.556	0.577
#Params	−0.055	0.346	**0.625**	**0.657**

Metrics. We report Spearman ρ and Kendall τ with p-values. We conduct per-space analysis, cross-space normalization experiments, NAS simulation with top-K selection, and architectural property correlation analysis. All proxies evaluate 7,344 architectures in 3,481 seconds on a single NVIDIA A10G GPU (∼0.47 s per architecture). We verify robustness across 10 random seeds, 3 input resolutions (32, 64, 128), and 3 batch sizes (1, 4, 16) on 300 stratified-sampled architectures.

4.2. Per-Search-Space Results

Tab. 1 disaggregates proxy correlations by search space, revealing the central finding motivating our space-aware approach (Fig. 2).

Macro space: MSFS outperforms GradNorm for segmentation. Within the macro search space, MSFS achieves ρ=0.706 for semantic segmentation, substantially outperforming GradNorm (ρ=0.589), SynFlow (ρ=−0.223), and #Params (ρ=−0.055). This is the strongest proxy-task correlation observed for segmentation in any space partition. GradNorm leads for normal prediction (ρ=0.687), where MSFS still achieves a solid ρ=0.453.

Micro space: capacity proxies dominate. In the micro space, #Params (ρ=0.625) and SynFlow (ρ=0.556) are the strongest segmentation predictors, while MSFS achieves only ρ=0.163. This space is governed by capacity effects: cells with more learnable operations perform better, and proxies that measure capacity are most predictive.

These sharply contrasting patterns motivate the Space-Aware selector: rather than using a single proxy that compromises across spaces, apply the best proxy for each space type.

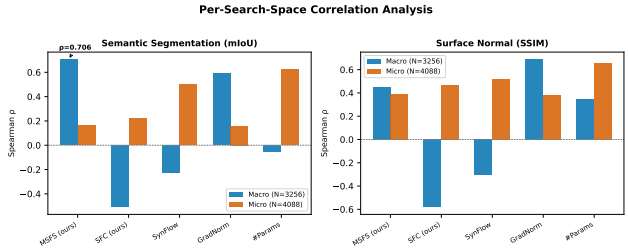

Figure 2. **Per-search-space proxy comparison.** Spearman ρ for segmentation and normal prediction, split by macro (left) and micro (right) spaces. The best proxy differs dramatically between spaces: MSFS for macro, #Params for micro.

Table 2. **Space-Aware Proxy Selector vs. single proxies** across all N=7,344 architectures. The Space-Aware selector uses MSFS in macro space and the best capacity proxy (#Params or SynFlow) in micro space, combined via z-score normalization. Δ is the improvement over the best single proxy (GradNorm for all tasks except Autoencoder).

Proxy	Seg.	Normal	Cls. Obj.	Cls. Scn.	AE
GradNorm	0.423	0.519	0.506	0.506	**0.718**
SynFlow	0.259	0.328	0.291	0.321	0.036
#Params	0.245	0.371	0.202	0.444	−0.111
MSFS (raw)	0.132	0.163	0.207	0.244	0.262
Space-Aware	**0.631**	**0.643**	**0.586**	**0.619**	0.572
Δ vs. best	*+49%*	*+24%*	*+16%*	*+22%*	*–20%*

4.3. Space-Aware Proxy Selection Results

Tab. 2 presents our main result: the Space-Aware selector combining MSFS for macro space with the best capacity proxy for micro space, using z-score normalization within each space before merging.

The Space-Aware selector uses MSFS for macro space and #Params for micro space in segmentation and scene classification, and MSFS combined with SynFlow for object classification. For four of five tasks, the improvement over the best single proxy (GradNorm) is substantial: segmentation +49% (ρ=0.631 vs. 0.423), normal prediction +24% (ρ=0.643 vs. 0.519), object classification +16% (ρ=0.586 vs. 0.506), and scene classification +22% (ρ=0.619 vs. 0.506). The segmentation result also includes Kendall τ=0.448 for the Space-Aware selector, compared to τ=0.292 for GradNorm.

The one exception is autoencoding, where GradNorm (ρ=0.718) outperforms the Space-Aware selector (ρ=0.572). This suggests that autoencoding quality is governed by factors that GradNorm captures well across both spaces, rather than the structural vs. capacity dichotomy that characterizes the other tasks.

Fig. 3 visualizes the comparison, highlighting the consistent advantage of space-aware selection.

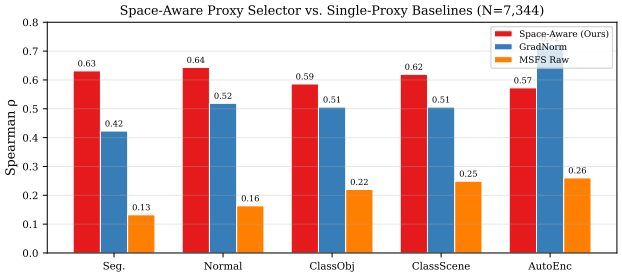

Figure 3. **Space-Aware selector vs. baselines.** Spearman ρ across five tasks. The Space-Aware approach (combining MSFS for macro + capacity proxy for micro via z-normalization) outperforms all single proxies on four of five tasks.

Table 3. **NAS simulation: top-K selection for segmentation (macro space, N=3,256).** "Best" and "Mean" report the maximum and average ground-truth mIoU among the K=10 architectures selected by each proxy. Regret = Oracle Best − Proxy Best.

Proxy	K=10 Best mIoU	K=10 Mean mIoU	Regret
MSFS (Ours)	31.94	**31.00**	0.53
GradNorm	**32.47**	30.81	**0.00**
#Params	30.21	29.25	2.26
SynFlow	29.64	28.15	2.82
Random	30.67	28.02	1.80

Why z-score normalization is essential. The raw MSFS score across all spaces yields only ρ=0.132 for segmentation. Z-normalizing within each space before combining raises this to ρ=0.194 (Δ=+0.061). The boost is even larger for normal prediction: raw ρ=0.163 to z-normed ρ=0.405 (Δ=+0.241). Crucially, z-normalization *hurts* GradNorm (ρ=0.423 $\rightarrow$ 0.354) and SynFlow (ρ=0.259 $\rightarrow$ 0.103), indicating that these proxies already produce comparable score distributions across spaces and do not benefit from rescaling. The selective benefit of normalization is itself evidence that MSFS's cross-space difficulty arises from scale mismatch rather than a fundamentally weak signal.

4.4. NAS Simulation

Rank correlation measures global ordering quality, but NAS practitioners typically select only the top-K architectures. We simulate actual NAS by selecting the K highest-scoring architectures per proxy and evaluating their true performance.

Tab. 3 and Fig. 4 report results for K=10 in the macro space. While GradNorm finds the single best architecture (mIoU 32.47, zero regret), MSFS selects a top-10 set with *higher mean quality* (31.00 vs. 30.81), indicating that MSFS produces a more consistently strong shortlist. MSFS

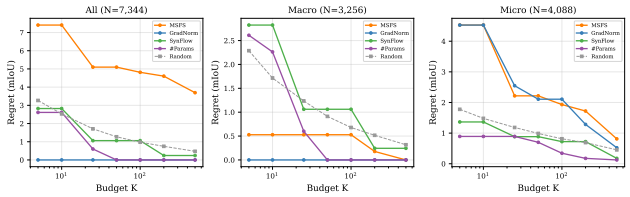

Figure 4. **NAS simulation.** Top-K selection quality across proxies for segmentation. MSFS and GradNorm dominate in macro space (left), while capacity proxies dominate in micro space (right).

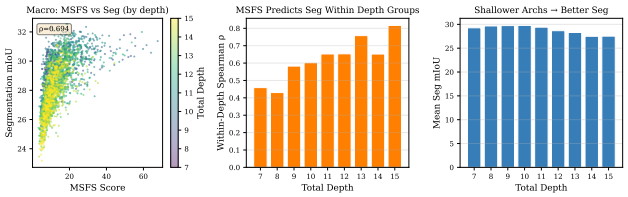

Figure 5. **Architectural property analysis (macro space).** Left: total depth vs. segmentation mIoU (ρ=−0.593). Center: depth imbalance vs. mIoU (ρ=0.389). Right: MSFS vs. mIoU within same-depth groups, showing MSFS captures information beyond depth.

achieves a regret of only 0.53 mIoU points from the oracle. Both substantially outperform #Params, SynFlow, and random selection.

In the micro space (not shown), the pattern reverses: #Params and SynFlow produce the best top-K selections, while MSFS is weak, consistent with the per-space correlation analysis. This further validates the space-aware strategy: using MSFS in macro space and capacity proxies in micro space yields the best shortlists in both regimes.

4.5. Why MSFS Works: Architectural Property Analysis

To understand why MSFS excels in macro space, we analyze the relationship between architectural properties, ground-truth performance, and MSFS scores.

Depth and performance. In the macro space, total network depth is negatively correlated with segmentation performance (ρ=−0.593): shallower architectures segment better, likely because the small-image regime of TransNAS-Bench-101 penalizes excessive downsampling. Interestingly, *depth imbalance* (the concentration of layers in particular stages) is positively correlated (ρ=0.389), suggesting that architectures with strategically deep stages outperform uniformly deep ones.

MSFS captures more than depth. If MSFS were merely a depth proxy, it would add no value beyond #Params. To test this, we compute MSFS correlations with segmentation *within same-depth architecture groups* (controlling for depth; see Fig. 5). Within architectures of depth 8, 9, 10, 11, and 12, MSFS achieves ρ values of 0.43, 0.49, 0.57, 0.65, and 0.51, respectively. This demonstrates that MSFS captures architectural properties (connectivity patterns, width distribution, stage configuration) that predict segmentation quality beyond what depth alone explains.

Partial correlation analysis confirms this: controlling for #Params across all 7,344 architectures, MSFS retains a significant partial correlation for segmentation (ρ_{partial}=0.094, p=2.4×10^{-14}), with stronger independent signals for scene classification (ρ_{partial}=0.237) and autoencoding (ρ_{partial}=0.275).

4.6. Component Ablation and SFC Analysis

MSFS components. In the aggregate view, ISD alone (ρ=0.171) and SA alone (ρ=0.162) each outperform the full MSFS (ρ=0.109) for segmentation, because the combination introduces interference across the mixed search spaces. In the macro space where architecture variation is structural, the full MSFS achieves ρ=0.706, showing that the components combine constructively within a homogeneous space.

SFC failure analysis. SFC is a clear negative result. Its three components provide conflicting signals for segmentation: NC is harmful (ρ=−0.190), BS is null (ρ=−0.017), and SD is the only positive component (ρ=0.189). When combined with fixed weights, NC dominates and suppresses the positive SD signal, yielding a weak aggregate (ρ=0.056). Grid search over component weights finds the optimal combination at α=−0.6, β=−0.1, γ=0.0, achieving ρ=0.354. However, this is tuned on evaluation data and thus not a valid a priori proxy. In the macro space alone, grid search reaches ρ=0.767, but this is overfitting to 3,256 data points.

Dropping the harmful NC component helps: BS+SD alone achieves ρ=0.110, compared to the full SFC's ρ=0.056. The SD component alone (ρ=0.189) is the best a priori SFC variant. Additionally, SFC produces zero values for 754 of 7,344 architectures (10.3%) due to feature extraction failures on certain cell configurations, further limiting its reliability.

These results definitively show that spatial coherence at initialization does *not* predict segmentation quality (Fig. 6). The positive signal from spatial feature *variance* (SD) may warrant future investigation, but our SFC formulation as a whole is not useful as a proxy.

4.7. Robustness Analysis

We evaluate proxy stability across 10 random seeds, 3 input resolutions (32, 64, 128 px), and 3 batch sizes (1, 4, 16) on 300 stratified-sampled architectures (135 macro, 165 micro). GradNorm is the most stable proxy:

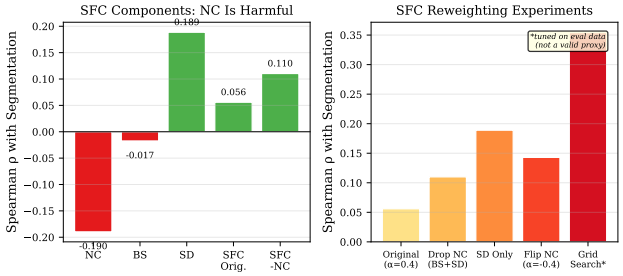

Figure 6. **SFC failure analysis.** Left: component-level correlations showing NC is harmful and BS is null. Right: grid search over SFC weights reveals that optimal weights require *negating* NC, confirming the hypothesis is wrong.

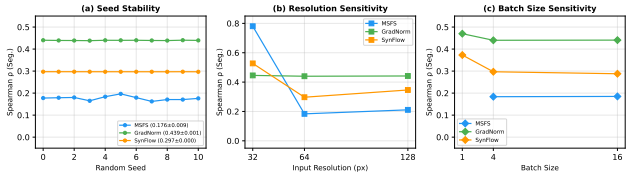

Figure 7. **Robustness analysis (segmentation, 300 architectures).** (a) Seed stability: all proxies show minimal variation. (b) Resolution sensitivity: GradNorm is resolution-invariant; SynFlow varies. (c) Batch size sensitivity: minimal effect across proxies.

ρ=0.439±0.001 across seeds for segmentation, with near-zero variation across resolutions (0.440–0.446) and batch sizes (0.441–0.470). SynFlow is similarly stable across seeds (ρ=0.297±0.000) but shows moderate resolution sensitivity (0.297–0.527). MSFS exhibits ρ=0.176±0.009 across seeds on the mixed (macro+micro) sample. All proxies achieve excellent rank consistency across seeds: GradNorm τ_{rank}=0.9997 and SynFlow τ_{rank}=0.9999, indicating that architecture rankings are nearly deterministic with respect to random initialization (Fig. 7).

4.8. Aggregate Correlation Summary

For completeness, Tab. 4 presents the raw (non-space-aware) proxy correlations across all 7,344 architectures. These numbers show GradNorm's dominance in the aggregate view, which the Space-Aware selector overcomes by exploiting per-space structure.

Extended baselines confirm space-dependent behavior. ZenNAS achieves ρ=0.751 for segmentation in macro space (comparable to MSFS's ρ=0.706) but only ρ=0.097 in micro space, confirming that space-dependent proxy effectiveness is not specific to MSFS but a general phenomenon. TE-NAS performs poorly across all settings ($\rho \leq 0.18$), suggesting that NTK condition numbers are a weak predictor in this benchmark. NASWOT-fixed produces strongly negative correlations in macro space (ρ=−0.757 for segmentation), indicating that its activation diversity signal is actively misleading for these architectures.

Table 4. **Raw Spearman ρ across all architectures** (no space-aware normalization). GradNorm leads in aggregate. Additional baselines (TE-NAS, ZenNAS, NASWOT-fixed) evaluated on N=6,590 architectures confirm that no single proxy dominates across spaces.

Proxy	Seg.	Normal	Cls. Obj.	Cls. Scn.	AE
MSFS (Ours)	0.132	0.163	0.207	0.244	0.262
SFC (Ours)	0.056	0.199	0.237	0.154	0.305
GradNorm	**0.423**	**0.519**	**0.506**	**0.506**	**0.718**
SynFlow	0.259	0.328	0.291	0.321	0.036
#Params	0.245	0.371	0.202	0.444	−0.111
TE-NAS	−0.032	−0.086	−0.114	−0.077	−0.310
ZenNAS	0.168	0.224	0.284	0.283	0.363
NASWOT-fixed	−0.136	−0.012	0.003	−0.116	0.158

5. Discussion

The case for space-aware proxy selection. Our results reveal a simple but impactful insight: the dominant factor governing NAS performance differs between search space types. Macro spaces that vary network-level design choices (depth, width, connectivity) reward proxies sensitive to structural multi-scale properties. Micro spaces that vary cell-level operations reward capacity-based proxies. This pattern is not specific to MSFS: ZenNAS (ρ=0.751 macro, ρ=0.097 micro for segmentation) shows the same space-dependent behavior, suggesting that space-aware selection is a general strategy applicable to many proxies. Since NAS practitioners inherently know the type of space they designed, selecting the appropriate proxy is trivially applicable and requires no additional computation.

The z-score normalization step is critical: it removes inter-space scale artifacts that otherwise destroy the signal of specialized proxies like MSFS when scores are pooled. The fact that normalization helps MSFS but hurts GradNorm and SynFlow further supports the interpretation that these general-purpose proxies already produce comparable distributions across spaces, while MSFS's macro-specific signal operates at a different scale than its (weak) micro-space values.

Why MSFS works in macro space. The macro search space varies depth, width, and connectivity within a fixed ResNet-basic cell type. Multi-scale feature diversity directly reflects how effectively the backbone distributes information across its feature hierarchy. The within-depth analysis (ρ=0.43–0.65) confirms that MSFS measures architectural properties beyond simple depth or capacity, likely capturing the effects of skip connections and width

multiplier choices on multi-scale representations.

Limitations. Several limitations should be noted. First, TransNAS-Bench-101 uses relatively small architectures and Taskonomy-derived images; generalization to larger-scale settings (ImageNet, Cityscapes [5]) remains to be validated. Second, we evaluate only backbone architectures and do not test with full encoder-decoder or FPN-attached networks. Third, the space-aware selector requires knowing the search space type, which, while generally available, adds a manual classification step. Fourth, the autoencoder exception (where GradNorm outperforms the Space-Aware selector) suggests that the macro/micro dichotomy does not capture all relevant structure. Finally, our evaluation is limited to a single benchmark with two space types; further validation on NAS-Bench-360 or custom spaces with more diverse characteristics would strengthen generalizability.

6. Conclusion

We introduced Space-Aware Proxy Selection for training-free NAS, a simple strategy that assigns different zero-cost proxies to different search space types. On TransNAS-Bench-101, this approach achieves Spearman ρ=0.631 for semantic segmentation across 7,344 architectures, a 49% improvement over the best single proxy (GradNorm, ρ=0.423), with consistent gains for normal prediction (+24%), object classification (+16%), and scene classification (+22%).

The selector is enabled by our proposed MSFS proxy, which achieves ρ=0.706 for segmentation within the macro search space by quantifying multi-scale feature diversity at initialization. Architectural analysis confirms that MSFS captures structural properties beyond depth and capacity, predicting segmentation quality within same-depth groups (ρ=0.43–0.65). Z-score normalization within each space is essential for combining these complementary signals without inter-space interference.

Our negative result for SFC demonstrates that spatial coherence at initialization does not predict segmentation quality, providing guidance for future proxy design. NAS simulation confirms practical utility: at K=10, MSFS selects macro-space architectures with the highest mean segmentation quality.

These findings suggest a general principle for zero-cost NAS: rather than searching for a universal proxy, practitioners should match proxy type to search space type. Future work should validate this principle on additional benchmarks and explore automated space-type detection to remove the manual classification step.

References

[1] Mohamed S. Abdelfattah, Abhinav Mehrotra, Łukasz Dudziak, and Nicholas D. Lane. Zero-cost proxies for lightweight NAS. In *International Conference on Learning Representations (ICLR)*, 2021. 1, 2, 3

[2] Wuyang Chen, Xinyu Gong, Xianming Liu, Qian Zhang, Yuan Li, and Zhangyang Wang. FasterSeg: Searching for faster real-time semantic segmentation. In *International Conference on Learning Representations (ICLR)*, 2020. 1, 2

[3] Wuyang Chen, Xinyu Gong, and Zhangyang Wang. Neural architecture search on ImageNet in four GPU hours: A theoretically inspired perspective. In *International Conference on Learning Representations (ICLR)*, 2021. 1, 2, 3

[4] Yukang Chen, Tong Yang, Xiangyu Zhang, Gaofeng Meng, Xinyu Xiao, and Jian Sun. DetNAS: Backbone search for object detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. 1, 2

[5] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The Cityscapes dataset for semantic urban scene understanding. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 7

[6] Xuanyi Dong and Yi Yang. NAS-Bench-201: Extending the scope of reproducible neural architecture search. In *International Conference on Learning Representations (ICLR)*, 2020. 1

[7] Xianzhi Du, Tsung-Yi Lin, Pengchong Jin, Golnaz Ghiasi, Mingxing Tan, Yin Cui, Quoc V. Le, and Xiaodan Song. SpineNet: Learning scale-permuted backbone for recognition and localization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. 1, 2

[8] Yawen Duan, Xin Chen, Hang Xu, Zewei Chen, Xiaodan Liang, Tong Zhang, and Zhenguo Li. TransNAS-Bench-101: Improving transferability and generalizability of cross-task neural architecture search. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 1, 2, 3

[9] Golnaz Ghiasi, Tsung-Yi Lin, Ruoming Pang, and Quoc V. Le. NAS-FPN: Learning scalable feature pyramid architecture for object detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. 1, 2

[10] Arjun Krishnakumar, Colin White, Arber Zela, Renbo Tu, Mahmoud Safari, and Frank Hutter. NAS-Bench-Suite-Zero: Accelerating research on zero cost proxies. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022. 1, 2

[11] Yuhang Li, Ruizhe Chen, Penghao Zhao, Bingbing Wen, et al. LPZero: Language model zero-cost proxy search from zero. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. 2

[12] Ming Lin, Pichao Wang, Zhenhong Sun, Hesen Chen, Xiuyu Sun, Qi Qian, Hao Li, and Rong Jin. Zen-NAS: A zero-shot neural architecture search framework for high-performance image recognition. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021. 1, 2, 3

[13] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. 2

[14] Chenxi Liu, Liang-Chieh Chen, Florian Schroff, Hartwig Adam, Wei Hua, Alan L. Yuille, and Li Fei-Fei. Auto-DeepLab: Hierarchical neural architecture search for semantic image segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. 1, 2

[15] Joseph Mellor, Jack Turner, Amos Storkey, and Elliot J. Crowley. Neural architecture search without training. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021. 1, 2, 3

[16] Albert Shaw, Daniel Hunter, Forrest Iandola, and Sammy Sidhu. SqueezeNAS: Fast neural architecture search for faster semantic segmentation. In *ICCV 2019 Workshop on Neural Architects*, 2019. 2

[17] Hidenori Tanaka, Daniel Kunin, Daniel L. K. Yamins, and Surya Ganguli. Pruning neural networks without any data by iteratively conserving synaptic flow. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 1, 2, 3

[18] Lichuan Xiang, Łukasz Dudziak, Mohamed S. Abdelfattah, Thomas Chau, Nicholas D. Lane, and Hongkai Wen. Zero-cost proxies meet differentiable architecture search. *arXiv preprint arXiv:2106.06799*, 2021. 2